

Agent/Workflow Task Orkestrasyonu

Altyapı Kararı — Airflow · Celery · Temporal · Mevcut Agent Engine

Karar analizi · 2026-08-09 · 4 aday × 6 eksen + 7 bilinen agent engine profili + Ek A

Bu belge ne içerir? Koçun brief'indeki 4 aday (Apache Airflow, Celery, Temporal ve mevcut brain_chat_V2 engine) altı eksenle karşılaştıran, karar-odaklı bir değerlendirme. Yönetici özeti + scorecard matrisi + her aday derinlemesine + **yedi bilinen agent engine'in task-yönetimi profili** (Hermes, OpenClaw, OpenCode, Codex, Claude Code + **Wren, Shannon**) + AI-ajan uyarıları + karar ağacı + öneri. **Ek A:** "task" kavramının katman/tuzak/statik-dinamik netleştirmesi.

Değerlendirme güncel resmî dokümanlarla; agent engine tarafı **yedi ajanın kaynak kodundan** incelendi (Wren ve Shannon bu istek üzerine klonlandı).

Agent/Workflow Task Orkestrasyonu — Altyapı Kararı

Airflow · Celery · Temporal · Mevcut Agent Engine (brain_chat_V2)

Amaç: Agent/workflow task'larının *oluşturulması, kuyruğa alınması, planlanması, çalıştırılması, retry edilmesi, durumunun takip edilmesi ve hata sonrası devam ettirilmesi* için hangi altyapının kullanılacağını belirlemek. **Karşılaştırma eksenleri:** task yönetimi · retry/recovery · state takibi · scheduling · concurrency · operasyonel karmaşıklık. **Adaylar:** Apache Airflow · Celery · Temporal · mevcut **brain_chat_V2** engine.

Not (brain_chat_V2): İç motorun kaynağı elimde olmadığından, onu "*güncel bir agent engine*" olarak, iki referans mimari üzerinden konumlandırımdı: **Hermes-Agent** (yerel kaynak kodundan birebir incelendi — SQLite üstü durable orchestrator) ve **LangGraph** durable-execution deseni (checkpoint/thread-resume). brain_chat_V2 bunlardan hangisine daha yakınsa ilgili satır onun için geçerlidir; kesin skor için kodunu §7'deki 6 soruyla eşleyebiliriz.

1. Yönetici özeti (önce karar)

Tek cümle: LLM-ajan task'ları için "doğru" altyapı, işin **süresine, exactly-once ihtiyacına ve ölçeğe** bağlı; tek bir kazanan yok — ama çoğu ajan yükü için **iki pratik yol** öne çıkıyor.

- İş uzun-süren, insan/harici-olay için duraklıyor, "tam-bir-kez" ve deterministik replay şartsa** → **Temporal**. Ajan dünyasındaki en güçlü dayanıklılık modeli; bedeli en yüksek öğrenme + operasyon eğrisi ve LLM adımlarını *activity*'e sarma disiplini.
- Mevcut engine zaten oturum/state yönetiyorsa ve tek ihtiyaç "dayanıklı kuyruk + task retry + crash-recovery" ise** → **iki seçenek:**
 - Buy (en hızlı):** kuyruk/retry/concurrency'i **Celery**'ye devret, workflow state'i engine'de tut.
 - Build (tam kontrol):** engine içine **Hermes-tarzı hafif orchestrator** (Postgres/SQLite `tasks+runs`, CAS-claim, lease+heartbeat, circuit-breaker, cron) — çekirdek ~1–2K satır, sıfır yeni servis.
- İş bir veri/batch DAG'ı, zamanlı tetik + operatör UI merkezdeyse** → **Airflow**.
- Sadece dağıtık task kuyruğu + worker havuzu + basit retry lazım, state'i kendin tutacaksan** → **Celery**.

Neden otomatik Temporal değil: LLM/tool çağrıları doğası gereği *non-deterministik*; Temporal'ın replay-determinizmini korumak için her yan-etkiyi activity'e sarmak ve idempotent yapmak gerekir. Aynı "resume'da adım baştan koşar → idempotent ol" tuzağı **LangGraph** ve **Airflow** için de geçerli (§5). Hermes'in kanıtladığı şey: tek makinede **SQLite + CAS-claim + lease + breaker**, ajan-task'larının büyük kısmı için Temporal garantilerinin *pratik* karşılığını çok daha ucuza verir.

2. Değerlendirme çerçevesi — 6 eksen ne demek

Eksen	Sorduğu soru
Task yönetimi	Task nasıl tanımlanır/oluşturulur/kuyruğa alınır? Tanım nerede yaşar (kod / DAG / DB satırı)?
Retry / recovery	Başarısızlıkta ne olur? Baştan mı, kaldığı yerden mi? Worker çökerse iş kaybolur mu? Exactly/at-least-once?
State takibi	Her işin durumu + geçmişi nerede? Denetlenebilir mi (audit/UI)?
Scheduling	Zamanlı (cron) + bağımlılıkla (DAG) tetik var mı? Backfill?
Concurrency	Paralellik nasıl? Yatay ölçek, worker havuzu, hız?
Operasyonel karmaşıklık	Kaç servis? Öğrenme eğrisi? Bakım yükü?

Kritik ayrım — iki retry katmanı: (a) *API-çağrısı retry* (429/5xx/timeout → backoff) hafiftir, hepsinde vardır. (b) *Task/workflow retry* (worker çöktü, iş kaldığı yerden/baştan sürsün) asıl zor olandır. Koçun sorduğu (b). Aşağıdaki tablo (b)'ye göre.

Kritik ayrım — hangi "task"? Bu belgede **task = bir İŞ/JOB (yaşam döngüsü olan iş birimi)**, tek bir tool çağrısı değil. Kelime her araçta farklı katmanı işaret ettiği için (Airflow/Celery'de "task" = adım; Temporal/agent-engine'de = iş) tüm scorecard iş (job) seviyesine normalize edilmiştir. Katman sözlüğü, terminoloji tuzağı, "yerleşik recovery" ve statik/dinamik ayrımı için → **Ek A**.

3. Scorecard — 4 aday × 6 eksen

Derece: ●●● güçlü · ●●○ orta · ●○○ zayıf/elde-değil.

Eksen	Airflow	Celery	Temporal	Mevcut Agent Engine (Hermes/ LangGraph-tarzı)
Task yönetimi	●●● DAG düğümü; Python	●●○ kuyruk + fonksiyon çağrısı	●●● workflow = kod (durable)	●●○ oturum/thread + (Hermes) DB <code>tasks</code> satırı
Retry / recovery	●●○ <code>retries</code> +delay, baştan replay; zombie reap	●●○ <code>max_retries</code> +backoff; <code>acks_late</code> , at-least-once (idempotency sende)	●●● kaldığı yerden replay, exactly-once activity, crash/gün-sonrası resume	●●○ Hermes: breaker+lease+crash-reclaim ●● / LangGraph: checkpoint-resume ●●
State takibi	●●● metadata DB + zengin UI	●○○ result backend; UI zayıf (Flower)	●●● event history + replay UI	●●○ Hermes: events+runs / LangGraph: checkpoint (StateSnapshot), UI değişken
Scheduling	●●● en güçlü : cron + veri-farkındalıklı + backfill	●●○ Celery Beat (basit cron)	●●○ Temporal Schedules (iyi)	●●○ Hermes: pluggable cron / LangGraph: yok (harici)

Concurrency	●●○ executor havuzları (Local/Celery/K8s) + pools	●●● native worker havuzu + prefetch, yüksek yatay ölçek	●●● task queue + worker fleet, sticky; çok yüksek ölçek	●●○ Hermes: dispatcher + swarm / LangGraph: süreç-içi
Operasyonel karmaşıklık	●○○ yüksek (scheduler+web+DB+executor+broker)	●●○ orta (broker+worker+backend+Flower)	●○○ yüksek (cluster/Cloud + SDK worker + determinizm disiplini)	●●● düşük (Hermes: tek süreç+SQLite / mevcut genişletme)

Okunuş:

- **Recovery kalitesi:** Temporal (kaldığı yerden, exactly-once) > Hermes-tarzı engine (crash-reclaim + breaker) ≈ Airflow/Celery-with-effort (baştan / at-least-once).
- **Scheduling:** Airflow açık ara önde.
- **Concurrency/ölçek:** Temporal ve Celery önde (indikatif: Temporal ~50–100K, Celery ~10–20K task/sn — kaynak: AI-orkestrasyon karşılaştırması).
- **Op. maliyeti:** mevcut engine'i genişletmek en ucuz; Airflow ve Temporal en pahalı.

4. Her aday — derinlemesine

4.1 Apache Airflow — "zamanlı DAG'ların kralı"

- **Task yönetimi:** İş, Python'da tanımlı **DAG** düğümleri (operatörler). Scheduler DAG'ları tarar, hazır task'ları executor'a dağıtır; durum metadata DB'de.
- **Retry/recovery:** Task başına `retries` + `retry_delay` (+ opsiyonel exponential backoff). **Kritik sınırlama:** retry, task'ı **kaldığı yerden değil baştan** çalıştırır. **Zombie** task'lar (heartbeat kaçınca) scheduler tarafından tespit edilip fail/retry edilir.
- **State:** Metadata DB (Postgres/MySQL) + **zengin görsel UI** (gözlemlenebilirlik en güçlü yanı).
- **Scheduling:** En güçlü — cron, veri-farkındalıklı tetik, **backfill/catchup**.
- **Concurrency:** Executor seçimi (Local/Celery/Kubernetes) + `pools` + `max_active_tasks/` `max_active_runs`.
- **Op. karmaşıklık:** Yüksek — scheduler + webserver + metadata DB + executor (+ CeleryExecutor için broker).
- **Ne zaman:** Veri/ETL batch pipeline'ları, gecelik işler, operatör görünürlüğü merkezde. **Ne zaman değil:** insan/harici-olay için saatlerce *duraklayan* uzun-süren adımlar (baştan-replay + scheduler modeli buna uygun değil).

4.2 Celery — "dağıtık task kuyruğu"

- **Task yönetimi:** Aktör modeli; **broker** (Redis/RabbitMQ) task'ları kuyruğa alır; task = fonksiyon çağırısı. `chain/group/chord` (Canvas) ile basit iş grafikleri.
- **Retry/recovery:** `max_retries` + `retry_backoff` + `autoretry_for`. **Varsayılan at-least-once** →

idempotency senin sorumluluğun. `acks_late=True` (iş bitince ack) + `task_reject_on_worker_lost`

kritik işler için şart. **Visibility timeout tuzağı:** Redis/SQS'te uzun süren task, timeout'u aşınca **yeniden teslim edilir** (timeout > en uzun task süresi olmalı).

- **State:** Opsiyonel **result backend** (Redis/DB); yerleşik UI zayıf (**Flower**).
- **Scheduling:** **Celery Beat** (basit cron).
- **Concurrency:** Native **worker havuzu** + prefetch; güçlü yatay ölçek; kısa-ömürlü işlerde çok verimli.
- **Op. karmaşıklık:** Orta — broker + worker'lar + result backend + Flower.
- **Ne zaman:** Bağımsız LLM inference işleri, hafif batch, mevcut Redis/RabbitMQ varsa, task başarısızlığı workflow'u zincirleme bozmuyorsa. **Ne zaman değil:** çok-adımlı, state-taşıyan, kaldığı-yerden-devam gereken ajan akışları (workflow state'i sen yönetmek zorunda kalırsın).

4.3 Temporal — "durable execution"

- **Task yönetimi:** Workflow = **kod** (Go/Java/TS/Python), config/DAG değil; yan-etkili adımlar **activity**.
- **Retry/recovery:** **En güçlü.** Her workflow **event-sourced history** olarak kaydedilir; worker çökerse history **replay** edilir, tamamlanmış activity'ler **atlanır** → **exactly-once activity completion**, **kaldığı yerden** devam. Activity başına `RetryPolicy` (otomatik); timer'lar günlerce/haftalarca dayanıklı.
- **State:** Event history (değişmez log) + **replay ile debug edilebilen Web UI** (tam denetlenebilirlik).
- **Scheduling:** Temporal **Schedules** (cron benzeri) — iyi.
- **Concurrency:** Task queue + **worker fleet** + sticky execution; çok yüksek ölçek.
- **Op. karmaşıklık:** Yüksek — **Temporal cluster** (self-host) ya da **Temporal Cloud** + her dilde SDK worker'ı. **Determinizm pazarlık dışı:** workflow kodunda rastgele değer / doğrudan saat / non-deterministik çağrı yasak; yan-etkiler **idempotent** olmalı.
- **Ne zaman:** Uzun-süren, çok-adımlı, insan/harici-olay bekleyen, mission-critical ajan workflow'ları (planner→search→writer→verifier). **Ne zaman değil:** "gecelik SQL export" gibi basit zamanlı işler (gereksiz ağırlık).

4.4 Mevcut Agent Engine (brain_chat_V2 ≈ Hermes / LangGraph-tarzı)

İki referans mimari, "güncel bir agent engine"nin bu işi nasıl çözdüğünü gösteriyor:

A) Hermes-Agent (yerel kaynak — SQLite üstünde tam durable orchestrator):

- **Task yönetimi:** `tasks` tablosu = 9-durumlu FSM (`triage→todo→ready→running→blocked→review→done`); `task_runs` = her **deneme**; `task_links` = **DAG** bağımlılık.
- **Retry/recovery:** **CAS-claim** (`UPDATE ... WHERE status='ready' AND claim_lock IS NULL` → dağıtık kilit olmadan **at-most-once** dispatch); **lease + heartbeat**; `detect_crashed_workers` (PID ölünce `ready`'ye geri); **circuit-breaker** (`consecutive_failures ≥ limit`, `per-task max_retries`); **tipli engel** (dependency/transient/needs_input/capability) ile akıllı yönlendirme; rate-limit exit'i failure saymaz.
- **State:** `task_events` (audit) + `task_runs` (deneme-deneme summary/error) → Temporal event-history'nin hafif karşılığı. **Resume:** run **summary** (handoff) ile kaldığı bağlamdan devam. **Idempotency:** `idempotency_key`.
- **Scheduling:** Pluggable **cron** provider. **Concurrency:** dispatcher + swarm topolojisi. **Op:** düşük — tek süreç + SQLite.

B) LangGraph deseni (durable execution):

- **Checkpoint**er her "super-step"te state'i (StateSnapshot) kalıcılaştırır; `thread_id` anahtar. Crash → **son checkpoint'ten devam**. `interrupt()` ile insan-döngüsü, saat-beklemeyle *aynı* primitiftir (4 gün de bekler, insanı da bekler).
- **Ortak tuzak (Temporal ile aynı)**: resume'da **düğüm baştan koşar** → `interrupt()` öncesi ücretli API/DB yazımları **idempotent** olmalı.

Sonuç: "Mevcut agent engine" ekseninde güçlü olan taraf — düşük operasyonel maliyet, oturum/state'in zaten yönetiliyor olması ve insan-döngüsünün doğal desteği. Zayıf/belirsiz olan — durable-kuyruk + crash-recovery + exactly-once **olgunluğu implementasyona bağlı** (Hermes-tarziysa güçlü; saf checkpoint-only ise "baştan-koşar + idempotency sende" seviyesinde).

4.5 Güncel agent engine manzarası — beş ajanın task-yönetimi profili

brain_chat_V2'nin "doğru hedefini" seçmek için, kaynak kodunu incelediğimiz beş güncel agent engine'i aynı 6 eksenle haritaladım. Bu, "mevcut bir agent engine" etiketinin **tek bir nokta değil, bir yelpaze** olduğunu gösteriyor — bir uçta Hermes gibi tam durable orchestrator, diğer uçta Claude Code gibi saf interaktif döngü.

Eksen	Hermes	OpenClaw	OpenCode	Codex	Claude Code
Task yönetimi	SQLite <code>tasks</code> FSM + DAG (<code>task_links</code>)	SQLite state/kuyruk/ registry; session- key ağaç + focus/ routing	in-process session ağacı + <code>Task</code> tool (fg/bg subagent)	(yerel) rollout oturumu · (bulut) managed task API	in-process döngü + subagent
Retry / recovery	●●● breaker + lease + PID crash- reclaim	●●○ gateway restart-recovery + terminal-outcome (ok/error/timeout/ failed)	●○○ API-backoff + git-snapshot revert; bg-job in-memory (restart'ta kaybolur)	●●○ (yerel) HTTP- backoff · (bulut) <code>AttemptStatus</code> retry	●○○ API- backoff; task- retry yok
State takibi	●●● <code>task_events</code> + <code>task_runs</code>	●●○ SQLite (checkpoint/cursor)	●○○ mesaj + git- snapshot	●●○ rollout JSONL + <code>state_db</code> · (bulut) <code>TaskStatus</code>	●○○ oturum + Pre/ PostCompact hook
Scheduling	●●○ pluggable cron	●●○ cron (provider/mock)	●○○ yok	●○○ yok (bulut tetik)	●○○ yok
Concurrency	●●○ dispatcher + swarm	●●○ subagent registry + routing	●●○ bg-job / subagent (depth- limit)	●●○ worker + bulut fleet	●●○ subagent context izolasyonu
Op. karmaşıklık	●●● düşük (tek süreç + SQLite)	●●○ düşük-orta (gateway + SQLite)	●●● düşük (CLI)	●●○ orta (yerel + bulut)	●●● düşük (CLI)

Ajan notları (task-yönetimi açısından):

- **OpenClaw** — Beklenenden olgun. `AGENTS.md` kuralı: *runtime state / kuyruk / registry / checkpoint / cursor için varsayılan depolama yalnızca SQLite*. Gateway yeniden başlarken **restart-recovery** koordinatörü çalışır (`subagent-registry-restart-recovery-coordinator`, `RecoveryRetry`, `resumeAcceptedRecovery` / `abandonSubagentRestartRecoveryLaunch`). `agent-run-terminal-outcome` wait/liveness/timeout'u **sticky terminal outcome**'a normalize eder (Hermes'in run-outcome taksonomisinin muadili). → Durable-tarapta Hermes'ten sonra en güçlü ikinci profil.
- **Codex** — İki katmanlı ve **hibrit build/buy**. (yerel) `rollout` (JSONL + `state_db` SQLite index) oturumu resume eder; `retry.rs` HTTP backoff+jitter. (bulut) `cloud-tasks` = **managed task backend'e delege**: `TaskStatus` (Pending → Ready → Applied) + `AttemptStatus` (Pending /

Completed / Failed / Cancelled) = gerçek bir managed yaşam döngüsü + retry. → "Ağır dayanıklılığı managed servise devret" felsefesi (Temporal Cloud'a benzer düşünce).

- **OpenCode** — Saf **in-process** kodlama ajanı. `Task` tool'u fg/bg subagent çalıştırır ama `BackgroundJob` **bellek-içi** (süreç ölürse iş kaybolur — durable değil). Retry yalnız **API-çağırısı** seviyesinde (backoff); "recovery" = **git-snapshot** ile dosya durumunu geri alma + `task_id` ile oturum resume. Durable kuyruk / cron / circuit-breaker **yok**.
- **Claude Code** — İnteraktif CLI. Büyük yan-işi **subagent'a** izole eder (ayrı context, ana pencereye özet). Task-seviyesi durable retry / crash-recovery / kuyruk **yok**; dayanıklılık kapsam dışı (tek-kullanıcı aracı).

brain_chat_V2 için çıkarım: Bu beş engine, dayanıklılık için **iki kanıtlanmış rota** gösteriyor —

1. **Kendi durable çekirdeğini kur** (Hermes / OpenClaw): SQLite üstünde `tasks+runs`, CAS-claim/lease, breaker, restart-recovery. Sıfır yeni servis, tam kontrol.
2. **Ağır dayanıklılığı managed backend'e devret** (Codex cloud): kuyruk/retry/lifecycle'ı dışarıdan al (Temporal/Celery bu rotanın altyapıları).

Saf **in-process/checkpoint-only** yol (OpenCode / Claude Code / LangGraph) hızlı ve ucuzdur ama "worker çökerse iş kaybolmasın" garantisini **tek başına vermez** — `brain_chat_V2` bu uçtaysa ve dayanıklılık isteniyorsa, yukarıdaki 1 veya 2'ye taşınması gerekir.

4.6 Bilinen agent'lar bu işi nasıl yapıyor — genişletilmiş manzara (Wren, Shannon)

"Bilinen agent'lar bu sistemi nasıl yapıyor" sorusu için iki tanınmış ajanı daha klonlayıp inceledim (kaynak kodundan). İkisi manzaranın iki ucunu net gösteriyor.

Wren AI — GenBI / text-to-SQL ajanı (kaynak: `Canner/WrenAI`)

- **Task modeli:** bir doğal-dil sorusu → **LangGraph** ReAct ajanı (`create_react_agent` + `ToolNode`, `langgraph>=1.0`) + **wren-engine** (MDL semantik katman → deterministik SQL üretimi/çalıştırma).
- **Task-yönetimi:** LangGraph'ın **checkpoint/thread** modelini miras alır (in-process, checkpoint-resume). "wren-engine" ise **stateless, deterministik sorgu motoru** — task orchestrator değil, A-seviyesi işi yönetmez.
- **Konum:** **in-process / checkpoint** ucu. Durable kuyruk / cron / crash-recovery **yok**; dayanıklılık checkpoint'era bağlı. → OpenCode/Claude Code ile aynı kategori, üstüne deterministik bir sorgu-yürütme katmanı.

Shannon — production agent framework (kaynak: `Kocoro-lab/Shannon`)

- **Task modeli:** agent loop'u **doğrudan Temporal üstünde** koşar — Go `orchestrator` servisi Temporal workflow/activity'leriyle çalışır (compose'da `temporalio/auto-setup` + `temporalio/ui`; kodda `workflow.ExecuteActivity`).
- **Task-yönetimi:** Temporal'ın tüm dayanıklılığını **hazır alır** — durable execution, otomatik retry, **insan-onayı workflow'ları** (`ApprovalManager` + approval request/resolved = Temporal signal ile **durable bekleme**), **time-travel debugging** (= event-history replay). Üstüne multi-strategy orchestration, swarm, token-bütçe kontrolü.
- **Konum:** "ağır dayanıklılığı **durable/managed motora devret**" rotasının (§4.5 rota #2) **canlı açık-kaynak kanıtı** — ve seçtiği motor **Temporal**. `brain_chat_V2` "buy" yolunu düşünüyorsa Shannon birebir referans mimari.


```
recovery" mı?
|   |   | En hızlı benimseme —————> CELERY (kuyruğu devret, state engine'de)
|   |   | Tam kontrol / sıfır servis —> BUILD: Hermes-tarzı hafif orchestrator (Postgres/
SQLite)
|   |   | Sadece dağıtık kuyruk + worker havuzu + basit retry mi?
|   |   |   |   |   | EVET —————> CELERY
```

7. brain_chat_V2 için öneri + sonraki adım

Öneri (varsayımsal, kodu görünce kesinleşir): brain_chat_V2 bir agent engine olarak muhtemelen oturum/mesaj state'ini zaten yönetiyor. O hâlde en düşük-riskli yol:

- **Kısa vade (buy):** Dayanıklı kuyruk + retry + concurrency'i **Celery**'ye devret; `acks_late` + `task_reject_on_worker_lost` + `visibility_timeout > max_task` + idempotent task'lar. Workflow state'i engine'de kalır.
- **Orta vade (build, önerilen hedef):** Engine içine **Hermes-tarzı hafif durable orchestrator** göm — Postgres/SQLite `tasks`+`task_runs` tabloları, CAS-claim, lease+heartbeat, `consecutive_failures` breaker, pluggable cron. Sıfır yeni servis bağımlılığı, tam kontrol; Hermes bunun **tam çalışan referans implementasyonu**.
- **Temporal'a geç** ancak: çok-makineli, yüksek-hacimli, katı exactly-once **veya** günlerce insan/harici-olay bekleyen workflow ihtiyacı netleştiğinde.

Kesin skor için brain_chat_V2'de şu 6 soruyu cevaplayalım (bu dökümanı ona göre güncellerim):

1. Task **kalıcı mı** saklanıyor (DB tablosu) yoksa süreç-belleğinde mi?
2. Worker çökerse iş **ne oluyor** — kayboluyor mu, devralınıyor mu?
3. Retry **hangi seviyede** — API çağırısı mı, tüm iş mi? Sayaç/limit var mı?
4. **Idempotency** ve **exactly/at-least-once** garantisi ne?
5. Zamanlı tetik (**cron**) ihtiyacı var mı, ne sıklıkta?
6. Beklenen **concurrency** (kaç paralel iş) ve **süre** (saniye mi, saat mi)?

Ek A — "Task" tam olarak ne demek? (kavramsal netleştirme)

"Task" aşırı yüklü bir kelime; her sistem başka bir şeye "task" der. Bu ek üç şeyi netleştirir: (A.1) hangi katmandan bahsediyoruz, (A.2) "yerleşik recovery" ne demek, (A.3) neden ajan task'ı "statik" değil.

A.1 — Üç granülerlik + terminoloji tuzağı

Her şeye "task" denebildiği için üç katmanı ayırmak şart. **Bu belgede "task" = A (iş/job).**

Katman	Ne	Örnek	Sistemlerde adı
A — İş / hedef	Yaşam döngüsü olan iş birimi (koçun "task"ı)	"auth modülünü refactor et"	Hermes kart (<code>tasks</code> satırı) · Airflow DAG-run · Temporal Workflow · Celery: yok (Canvas ile elle)
B — Adım	Tek tool çağırısı / LLM turu	<code>read_file(auth.py)</code>	Airflow task (DAG düğümü) · Temporal Activity · Celery task

C — Alt-ajan	A'nın bir parçası için spawn edilen alt-iş	"40 dosyayı tara"	Hermes <code>delegate_task</code> /swarm · OpenCode/Claude Code <code>Task tool</code>
---------------------	--	-------------------	--

Tuzak: Kelime, adaylarda **doğal olarak farklı katmanı** işaret eder:

- **Airflow / Celery dokümanlarında "task" = B (adım).** Airflow'da bir DAG düğümü, Celery'de bir fonksiyon çağrısı "task"tır. A seviyesi Airflow'da "DAG run", Celery'de **yok** (Canvas ile sen kurarsın).
- **Temporal'da kullanıcı-seviyesi = Workflow (A) + Activity (B);** "task queue / workflow task" düşük-seviye motor terimidir — koçun "task"ıyla karıştırma.
- **Hermes / agent engine'de "task" = A** (Kanban kartı) — tam koçun kastettiği anlam.

Sonuç: "Airflow task retry yapar" → **bir adımı (B)** baştan koşar. "Temporal task'ı kaldığı yerden sürdürür" → **bütün işi (A)** kurtarır. **Aynı kelime, farklı katman.** Scorecard hep A'ya göre yazıldı.

Tool-trace ile ilişki (iki ayrı katman): *Tool-trace compaction* = bir A-task'ının **içindeki adımları (B)** yönetmek (context penceresini sıkıştırmak). *Task management* = **A-task'ının kendisini** yönetmek (yaşam döngüsü, kuyruk, retry, crash-recovery). Biri "tool çıktıları context'e sığsın", öbürü "iş worker çökse de kaybolmasın".

A.2 — "Yerleşik A-recovery" vs "B-retry verir; A'yı sen inşa edersin"

Recovery = **worker için ORTASINDA çökerse ne olur?** İş (A) = "auth refactor", adımlar: 1) `read_file` ✓ → 2) `run_tests` ✓ → 3) `write_file` ⌚ ← **worker burada çöktü** → 4) `run_tests` (başlamadı).

- **Yerleşik (Temporal, Hermes):** Sistem, işin 3. adımda olduğunu ve 1–2'nin bittiğini **kalıcı olarak bilir**; çökmeyi kendi fark eder, işi başka worker'a devredip **3'ten** sürdürür. Temporal: event-history'yi replay eder, biten activity'leri atlar. Hermes: task `running`+lease; PID ölünce/lease dolunca `ready`'ye döner, başka worker claim edip önceki denemenin **özetinden (handoff)** devam eder. **Sen kurtarma kodu yazmazsın.**

- **B-retry, A'yı sen inşa edersin (Airflow, Celery):**

- **Celery — tamamen elle:** "task" = tek fonksiyon; onu retry eder ama **çok-adımlı iş kavramı yok**. Canvas `chain` ile kurarsın; worker ortada çökerse mesaj yeniden teslim edilir (at-least-once → idempotency senin), ama **"iş 3. adımdaydı" bilgisini Celery saklamaz** — kendi DB'nde tutarsın.
- **Airflow — kısmen:** İş **statik DAG** olarak (ayrı düğümler) yazarsan, Airflow biten düğümleri metadata DB'de tutar → çökünce **başarısız düğümden** devam eder (1–2'yi tekrarlamaz). **Ama:** (1) iş önceden **statik DAG** olmalı, (2) düğüm **içinde** checkpoint yok (7/10'da çökerse düğüm baştan koşar), (3) dinamik ajan döngüsü DAG'a sığmaz.

	Çökme sonrası "iş nerede kalmıştı" defterini kim tutar?	Sen ne yazarsın?
Temporal / Hermes	Sistem (event history / <code>tasks</code> +lease)	Sadece iş mantığı
Airflow	Sistem ama sadece statik DAG düğümleri arasında	İş statik DAG'a dök; düğüm-içi checkpoint yoksa sen ekle
Celery	Hiç kimse (sen)	İş-durumu DB'si + "nerede kaldım" + idempotency + çökme tespiti

A.3 — Klasik task statiktir; ajan task'ı DİNAMİktir

"Static" = **işin şekli (hangi adımlar, kaç tane, hangi dallar) koşmadan önce belli mi?** (Tanımın kodda önceden yazılması ayrı şey — o hepsinde böyle.)

- **Klasik (Airflow) = static:** grafik önceden çizilir, her koşuda aynı şekildir. Ör: "her gece extract → transform → load."
- **Ajan = dinamik:** grafik **koşarken ortaya çıkar**. İki düzeyde:
 1. **Adımlar (B) her zaman dinamik** — model her turda ne gördüğüne bakıp sıradaki tool'a karar verir. Ör: read → grep → run_tests → (patladı) → read_log → edit → run_tests → (hâlâ) → web_search → edit → ✓. Bu diziyi/dalları kimse önceden çizemez; test geçseydi web_search hiç olmayacaktı.
 2. **İşler (A) bile runtime'da doğabilir** — ajan koşarken **yeni task/alt-ajan** yaratır (Hermes create_task/delegate_task/swarm); bu task planda yoktu.

Sistem	İşin şekli ne zaman belli?	Dinamik + durable birlikte?
Airflow	Koşmadan önce (statik DAG)	✗ Model-kararlı serbest dallanma ifade edilemez
Temporal	Koşarken (workflow = kod: if/loop/dinamik activity)	✓ Kod aktıkça history'ye yazılır
Hermes / agent engine	Koşarken (tool çağrıları + runtime create_task)	✓ Her deneme + handoff kaydedilir

Not (dürüst nüans): Airflow'da **dynamic task mapping** (`.expand()`) vardır — bir koleksiyon üzerinde runtime'da paralel düğüm üretir (ör. "gelen 30 dosya için 30 task"). Ama bu **sınırlı, bildirimsel** bir genişletmedir; modelin "şimdi grep, sonra duruma göre ya web_search ya edit" gibi **serbest, sonuca-bağlı** kararlarını ifade edemez.

Neden kararın merkezinde: Ajan işi **dinamik ve çok-adımlı** olduğundan statik DAG'a sığmaz. "Dinamik ve kaldığı yerden devam" ikisini birden isteyen ajan yükü için doğru araçlar **Temporal** veya **Hermes-tarzı engine**'dir; Airflow'un statik-DAG modeli buna uymaz.

Kaynaklar

- **Airflow** — [Tasks \(Airflow 3.x docs\)](#) · [Task Retries & Retry Delays](#) · [Error Handling in Airflow](#)
- **Celery** — [Tasks \(Celery docs\)](#) · [Configuration & defaults](#) · [Task Resilience \(GitGuardian\)](#) · [Retries & Visibility Timeouts at Scale](#)
- **Temporal** — [Beyond State Machines \(temporal.io\)](#) · [Durable Execution makes workflows just work](#) · [Temporal vs Airflow \(ZenML\)](#)
- **AI-ajan orkestrasyonu** — [Orchestrating AI Tasks: Celery vs Temporal](#)
- **Agent engine desenleri** — [LangGraph Persistence \(LangChain docs\)](#) · [Human-in-the-Loop Agents in LangGraph](#) · **Hermes-Agent** yerel kaynak: `hermes_cli/kanban_db.py` (durable kernel)